BANKING MANAGEMENT SYSTEM

OOPS (PPF) Mini-Project Report

Prepared by: Farhan
Language: C++ (C++17)

1. Introduction

The Banking Management System is a menu-driven C++ application designed to manage savings and current bank accounts. It supports account creation, deposits, withdrawals, account details, balance display, validation, and persistent storage of account and transaction records.

2. Objectives

• Create savings and current accounts.

• Deposit and withdraw money from an account.

• Display account information and available balance.

• Reject withdrawals when the requested amount exceeds the available balance.

• Store account records and transaction history in text files.

3. Classes Used

Class	Purpose
Account	Abstract base class containing account number, holder name, balance, deposit, withdrawal and display operations.
SavingsAccount	Derived class representing a savings account; overrides account type and withdrawal operation.
CurrentAccount	Derived class representing a current account; overrides account type and withdrawal operation.
TransactionLogger	Appends deposit and withdrawal details to transactions.txt.
Bank	Manages account objects, menu operations, searching, loading and saving account records.

4. OOP Concepts Implemented

• **Classes and Objects:** Account-related classes define the structure and behaviour of bank accounts; Bank creates and manages their objects.

• **Encapsulation:** Account data members are protected and are accessed through public member functions.

• **Inheritance:** SavingsAccount and CurrentAccount inherit from the Account base class.

• **Runtime Polymorphism:** Virtual functions such as getAccountType(), withdraw() and display() are overridden and invoked through base-class pointers.

• **Constructors:** Constructors initialize account number, holder name and opening balance.

• **Exception Handling:** Invalid amounts, missing accounts, duplicate account numbers and insufficient balance are handled using exceptions.

• **File Handling:** accounts.txt stores account data; transactions.txt stores deposit and withdrawal records.

5. Program Flow

Start the program → load saved accounts from accounts.txt → show the menu → accept the user's choice → perform the selected operation → validate input and account rules → save updated account data and append transaction details when applicable → return to the menu → exit.

6. Sample Output

```
========== BANKING MANAGEMENT SYSTEM ==========
1. Create Savings / Current Account
2. Deposit Money
3. Withdraw Money
4. Display Account Details
5. Display All Accounts
0. Exit
Enter your choice: 1

Enter account number: 1001
Enter account holder name: Farhan
1. Savings Account
2. Current Account
Choose account type: 1
Enter opening balance: Rs. 5000
Account created successfully!

Enter your choice: 2
Enter account number: 1001
Enter deposit amount: Rs. 1500
Deposit successful. New balance: Rs. 6500.00

Enter your choice: 3
Enter account number: 1001
Enter withdrawal amount: Rs. 7000
Error: Insufficient balance. Withdrawal cancelled.
```

7. Test Cases

Test	Input / Action	Expected result
1	Create account with valid details	Account created and saved.
2	Deposit a positive amount	Balance increases and transaction is logged.
3	Withdraw an amount within balance	Balance decreases and transaction is logged.
4	Withdraw more than available balance	Withdrawal rejected; balance remains unchanged.
5	Deposit zero or a negative amount	Invalid amount rejected.
6	Search for an unknown account number	Account not found message.
7	Restart the application	Previously saved accounts are loaded.

8. Files Created During Execution

accounts.txt — stores account type, account number, holder name and balance. **transactions.txt** — stores account number, transaction type and amount. Both files are created in the program's working directory when the program runs.

9. Conclusion

The project demonstrates the required object-oriented programming concepts through a practical banking application. It provides basic account and transaction management, validates operations, prevents withdrawals beyond the available balance, and retains records using file handling.

10. Compilation and Execution

```
g++ -std=c++17 BankingManagementSystem.cpp -o BankingManagementSystem
./BankingManagementSystem   (Linux/macOS)
BankingManagementSystem.exe (Windows, after compilation)
```